



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

PRACTICA 5

Practica recursividad y factorización FINAL

COMPILADORES

5CM3

INTEGRANTES:

Díaz Torres Jonathan Samuel

Vázquez Blancas Cesar Said

PROFESOR:

ALCANTARA MENDEZ ALBERTO JESUS

FECHA DE ENTREGA: 26 DE OCTUBRE DE 2024

INSTITUTO POLITÉCNICO NACIONAL



Objetivo de la práctica

- ☐ Comprender cómo la recursividad por la izquierda y la falta de factorización afectan la implementación de una gramática en ANTLR4.
- ☐ Aplicar algoritmos de eliminación de recursividad por la izquierda y factorización para optimizar la gramática.
- ☐ Observar las diferencias en la implementación y ejecución antes y después de las transformaciones.

Instrucciones de la práctica:

- ☐ **Definir la gramática inicial** en ANTLR4 que contenga recursividad por la izquierda y no esté factorizada.
- ☐ **Ejecutar la gramática en ANTLR4 Lab** para observar cómo se comporta en el parseo.
- ☐ **Aplicar el algoritmo de eliminación de recursividad por la izquierda** a la gramática y reimplementarla en ANTLR4.
- ☐ **Aplicar el algoritmo de factorización** a la gramática y realizar la implementación final en ANTLR4.
- ☐ **Comparar** los resultados de cada versión de la gramática (original, sin recursividad por la izquierda y factorizada) en cuanto a su comportamiento y estructura de los árboles de parseo.

Gramática inicial (con recursividad por la izquierda y sin factorización)

Gramática original:

grammar Expresiones;

expr: expr '+' term // Recursividad por la izquierda

| expr '-' term // Recursividad por la izquierda

| term; term: 'a' | 'b' | 'c';

la parte donde seremos claros con el cliente pero respetando su privacidad y el que hará con su producto, ya que no tendremos datos de su empresa o de el mismo que sean importantes

1.-Implementa esta gramática en ANTLR4 Lab y prueba con expresiones de entrada como:

- $a + b - c$
- $a - b$
- $b + c + a$

Lexer Parser Sample ?

```
1  parser grammar ExprParser;
2  options { tokenVocab=ExprLexer; }
3
4  expr: expr '+' term
5  | expr '-' term
6  | term;
7
8  term: 'a'
9  | 'b'
10 | 'c';
```

Input sample.expr ?

```
1  b + c + a
```

Start rule ?

expr Run Show profiler

Tree Hierarchy

```
graph TD
    expr1["expr:1"] --- expr2["expr:1"]
    expr1 --- plus1["+"]
    expr1 --- term1["term:1"]
    expr2 --- expr3["expr:1"]
    expr2 --- plus2["+"]
    expr2 --- term2["term:1"]
    expr3 --- term3["term:1"]
    term3 --- b["b"]
    term2 --- c["c"]
    term1 --- a["a"]
```

Lexer Parser Sample ?

```
1 parser grammar ExprParser;  
2 options { tokenVocab=ExprLexer; }  
3  
4 expr: expr '+' term  
5 | expr '-' term  
6 | term;  
7  
8 term: 'a'  
9 | 'b'  
10 | 'c';
```

Input sample.expr ?

```
1 a + b - c
```

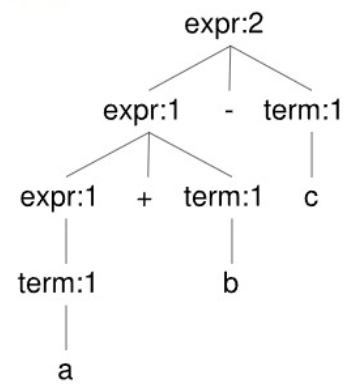
Start rule ?

expr

Run

Show profiler

Tree Hierarchy



```
1 parser grammar ExprParser;  
2 options { tokenVocab=ExprLexer; }  
3  
4 expr: expr '+' term  
5 | expr '-' term  
6 | term;  
7  
8 term: 'a'  
9 | 'b'  
10 | 'c';
```

```
1 a + b
```

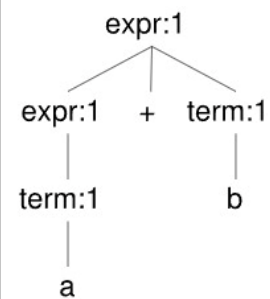
Start rule ?

expr

Run

Show profiler

Tree Hierarchy



Observa si ANTLR4 Lab muestra problemas de recursividad por la izquierda o construye el árbol de parseo correctamente.

ANTLR presenta la recursividad por la izquierda, construye el árbol de parseo bien, sin embargo el árbol se inclina a la izquierda

Eliminación de recursividad por la izquierda

- Realiza la eliminación de la recursividad por la izquierda.
- **Implementa esta versión de la gramática en ANTLR4 Lab** y vuelve a probar con las mismas expresiones. Observa cómo cambia el comportamiento y la estructura del árbol de parseo.

Lexer Parser Sample ?

```
1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 expr: term expr2 ;
5
6 expr2: '+' term expr2
7       | '-' term expr2
8       | ;
9
10 term: 'a'
11      | 'b'
12      | 'c';
```

Input sample.expr ?

```
1 b + c + a
```

Start rule ?
expr Run Show profiler

Tree Hierarchy

```
graph TD
    expr1["expr:1"] --> term1["term:1"]
    expr1 --> expr2_1["expr2:1"]
    term1 --> b["b"]
    expr2_1 --> plus1["+"]
    expr2_1 --> term2["term:1"]
    expr2_1 --> expr2_2["expr2:1"]
    term2 --> c["c"]
    expr2_2 --> plus2["+"]
    expr2_2 --> term3["term:1"]
    expr2_2 --> expr2_3["expr2:3"]
    term3 --> a["a"]
```

Lexer Parser Sample ?

```

1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 expr: term expr2 ;
5
6 expr2: '+' term expr2
7       | '-' term expr2
8       | ;
9
10 term: 'a'
11       | 'b'
12       | 'c';

```

Input sample.expr ?

```

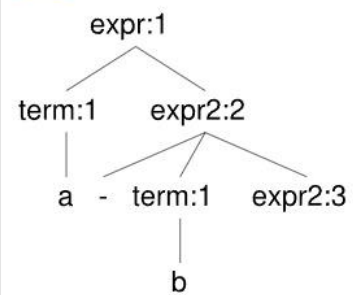
1 a - b

```

Start rule ?

expr Run Show profiler

Tree Hierarchy



Lexer Parser Sample ?

```

1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 expr: term expr2 ;
5
6 expr2: '+' term expr2
7       | '-' term expr2
8       | ;
9
10 term: 'a'
11       | 'b'
12       | 'c';

```

Input sample.expr ?

```

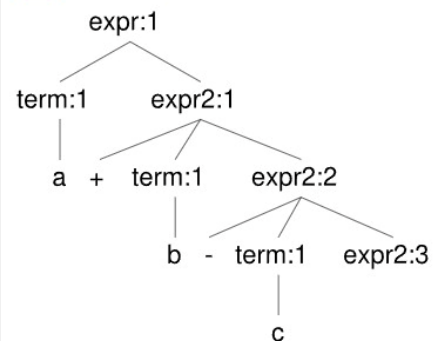
1 a + b - c

```

Start rule ?

expr Run Show profiler

Tree Hierarchy



El árbol de parseo ahora se inclina a la derecha, se desarrolla ahí, sin embargo, el orden y organización de este se ve afectado, creando y desorganizando algunos símbolos no terminales lo que hace que se vea un tanto desorganizado y con ramas donde ya debe de haberse visto símbolos terminales en la otra parte del árbol

Factorización

- Realiza la factorización por la izquierda de la gramática.
- **Implementa esta versión factorizada en ANTLR4 Lab** y realiza nuevamente las pruebas con las mismas entradas.

Lexer Parser Sample ?

```
1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 expr: term expr2 ;
5
6 expr2: '+' expr3
7       | '-' expr3
8       | ;
9
10 expr3: term expr2;
11
12 term : 'a'
13       | 'b'
14       | 'c';
```

Input sample.expr ?

```
1 b + c + a
```

Start rule ?
expr

Run Show profiler

Tree Hierarchy

```
graph TD
    expr1["expr:1"] --> term1["term:1"]
    expr1 --> expr2_1["expr2:1"]
    term1 --> b["b"]
    expr2_1 --> plus1["+"]
    expr2_1 --> expr3_1["expr3:1"]
    expr3_1 --> term2["term:1"]
    expr3_1 --> expr2_2["expr2:1"]
    term2 --> c["c"]
    expr2_2 --> plus2["+"]
    expr2_2 --> expr3_2["expr3:1"]
    expr3_2 --> term3["term:1"]
    expr3_2 --> expr2_3["expr2:3"]
    term3 --> a["a"]
```

Lexer Parser Sample ?

```
1 parser grammar ExprParser;  
2 options { tokenVocab=ExprLexer; }  
3  
4 expr: term expr2 ;  
5  
6 expr2: '+' expr3  
7       | '-' expr3  
8       | ;  
9  
10 expr3: term expr2;  
11  
12 term : 'a'  
13       | 'b'  
14       | 'c';
```

Input sample.expr ?

```
1 a - b
```

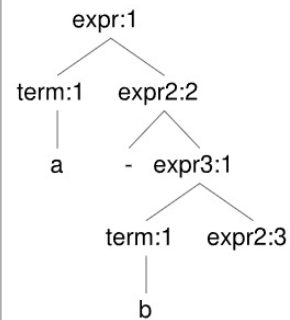
Start rule ?

expr

Run

Show profiler

Tree Hierarchy



Lexer Parser Sample ?

```
1 parser grammar ExprParser;  
2 options { tokenVocab=ExprLexer; }  
3  
4 expr: term expr2 ;  
5  
6 expr2: '+' expr3  
7       | '-' expr3  
8       | ;  
9  
10 expr3: term expr2;  
11  
12 term : 'a'  
13       | 'b'  
14       | 'c';
```

Input sample.expr ?

```
1 a + b - c
```

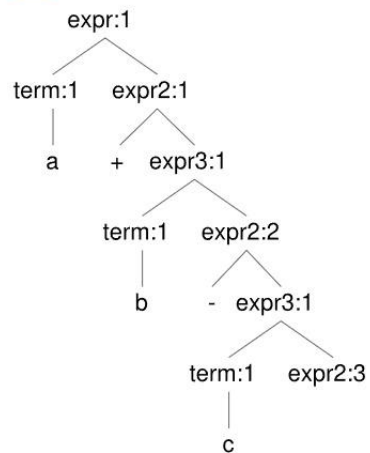
Start rule ?

expr

Run

Show profiler

Tree Hierarchy



El árbol de parseo ahora esta mas organizado y siguiendo las reglas debidas

Comparación de Resultados

- Estructura del árbol de parseo en cada caso.

En la primera parte teníamos organización pero la recursión por la izquierda era recurrente, en la segunda parte se eliminó la recursión por la izquierda pero a cambio perdimos un poco la organización del árbol, la tercera parte corrigió esto y dio un árbol mas limpio y puro

- Complejidad de la gramática antes y después de cada transformación.

En la primera parte era muy sencilla, la segunda se complico mas y en la tercer se vio mas organizada y limpia la gramática, sin embargo, se fueron aumentando líneas de la gramática lo que la hizo más larga

- Facilidad de implementación y resultados de ejecución.

La implementación fue buena y limpia, sin embargo la limpieza de la gramática tenia cosas un tanto complejas y que debían de pensarse para que quedara bien y se reflejaran los resultados, los resultados fueron los esperados y reflejaron lo que tenia de problemas cada caso y la importancia de estos métodos , ya que pasamos de una gramática con recursión por la izquierda a una con recursión por la derecha pero desorganizada y para arreglar este problema la factorización ayudo quedando un árbol limpio y organizado sin problemas